«module»
iam_service (cache + fail-safe)

+ IAM_CACHE_TTL_SECONDS = 1800
+ IAM_FAIL_CACHE_TTL_SECONDS = env IAM_FAIL_CACHE_TTL | 60
+ NIVELES_REVOCAN_SESION = frozenset({"critico"})

- _cache_key(uid: str): str
- _get_cached(uid: str): tuple[str | None, str | None] | None
- _store_cached(uid: str, role: str | None, department: str | None, ttl: float = IAM_CACHE_TTL_SECONDS): None
+ resolver_rol_departamento(uid: str): tuple[str | None, str | None]
- _sync_fail_key(uid: str): str
+ sincronizar_riesgo_iam(uid: str, nivel: str | None): bool

«usa» get_iam_provider().resolve / aplicar_riesgo

«Protocol»
IAMProvider

+ resolve(uid: str): tuple[str | None, str | None]
+ aplicar_riesgo(uid: str, nivel: str, revocar_sesiones: bool): bool

providers 1..*

«realiza» (estructural)

MockIAMProvider

+ resolve(uid: str): tuple[str | None, str | None] GET {IAM_MOCK_URL}/Users/{uid}
+ aplicar_riesgo(uid: str, nivel: str, revocar_sesiones: bool): bool (siempre False)

KeycloakIAMProvider

+ cfg: _KcCfg

+ __init__(cfg: _KcCfg | None = None)
- _buscar_usuario(uid: str, token: str): dict | None GET /admin/realms/{realm}/users?username=&exact=true
+ resolve(uid: str): tuple[str | None, str | None] attributes.role / attributes.department
+ aplicar_riesgo(uid: str, nivel: str, revocar_sesiones: bool): bool PUT /users/{id} (ueba_risk_level); POST /users/{id}/logout

EntraIAMProvider

+ cfg: _EntraCfg

+ __init__(cfg: _EntraCfg)
- _usuario(uid: str, token: str): dict | None GET graph /users/{uid}?$select=id,jobTitle,department
+ resolve(uid: str): tuple[str | None, str | None] (jobTitle, department)
+ aplicar_riesgo(uid: str, nivel: str, revocar_sesiones: bool): bool PATCH /users/{id} extensionAttribute1; POST /users/{id}/revokeSignInSessions

ChainIAMProvider (Composite)

+ providers: list[IAMProvider]

+ __init__(providers: list[IAMProvider])
+ resolve(uid: str): tuple[str | None, str | None] primer resultado ≠ (None, None) gana
+ aplicar_riesgo(uid: str, nivel: str, revocar_sesiones: bool): bool

cfg 1

«NamedTuple»
_KcCfg

+ base_url: str
+ realm: str
+ client_id: str
+ client_secret: str

cfg 1

«NamedTuple»
_EntraCfg

+ tenant: str
+ client_id: str
+ client_secret: str

«crea» (_build_provider, _from_item)

«module»
auth

+ SESION_TTL_SECONDS = 28800 (8 h)
+ PASSWORDS_PLACEHOLDER = ("TU_PASSWORD_ACA", "PLACEHOLDER_CARGAR_MANUALMENTE")
- _NO_APLICA = object()
env: SCORING_API_KEY, SCORING_ADMIN_KEY, SCORING_STRICT_AUTH, ADMIN_PASSWORD, OPERADOR_PASSWORD

- _eq(a: str, b: str): bool hmac.compare_digest
- _configured(name: str): str | None
+ strict_auth_enabled(): bool
+ keys_configuradas(): list[str]
+ admin_key_configurada(): bool
+ agent_key_configurada(): bool SCAN agent_key:*
- _agent_key_lookup(raw_key: str): dict | None
- _session_key(token: str): str
+ crear_sesion(role: str): str token_urlsafe(32)
- _rol_de_sesion(token: str): str | None
+ cerrar_sesion(token: str): None
+ require_api_key(x_api_key = Header(None)): None «Depends» 401/503
+ require_agent_key(x_api_key = Header(None)): dict | None 401/503
+ autorizar_agente(entrada: dict | None, uid: str, agent_id = _NO_APLICA): None 403
+ require_admin_key(x_api_key = Header(None)): None 403/503
- _password_configurada(var: str, valor_ejemplo: str): str | None
+ authenticate(username: str, password: str): str | None admin | operador

«module»
notifications (acción notificar_admin)

+ SMTP_TIMEOUT_SEC = 3
+ NOTIF_COOLDOWN_SEC = env | 600
+ GEMINI_API_URL = "https://generativelanguage.googleapis.com/v1beta/models/{modelo}:generateContent"
+ GEMINI_MODELOS = ["gemini-3.1-flash-lite", "gemini-2.5-flash"]
+ GEMINI_TIMEOUT_SEC = 6.0, GEMINI_MAX_REINTENTOS_429 = 1
+ MAX_CHARS_CAMPO = 80, MAX_EVENTOS_PROMPT = 10, MAX_CHARS_SALIDA = 1500, MIN_CHARS_SALIDA = 20
+ SALIDA_PROHIBIDA, ENCABEZADO_SALIDA: re.Pattern, IDENTIFICADOR_SNAKE: re.Pattern
+ TIPO_EVENTO_DESC, FEATURE_DESCRIPCIONES, DIRECCION_LEGIBLE: dict
+ GUIA_QUE_HACER_AHORA: list[str], GEMINI_REGLAS: str

- _sanear(valor, max_len: int = MAX_CHARS_CAMPO): str
- _formatear_guia_que_hacer_ahora(): str
- _hora_legible(timestamp_iso: str | None, con_segundos: bool = True): str | None
- _formatear_evento_reciente(evento: dict): str
- _formatear_eventos_recientes_desde(eventos: list[dict]): str
- _humanizar_feature(nombre_feature: str): str
- _formatear_explicacion(explicacion: list[dict] | None): str
- _fmt_umbral(valor: float): str
- _rango_nivel(nivel: str | None): str | None
- _armar_datos_gemini(uid: str, nivel: str | None, score: float | None, eventos: list[dict], explicacion: list[dict] | None): str
- _validar_salida(texto: str, finish_reason: str | None): str | None
- _generar_narrativa_gemini(uid, nivel, score, eventos, explicacion): str | None HTTPS REST
+ notificar_admin_si_corresponde(uid: str, nivel: str | None, score: float | None, explicacion: list[dict] | None = None, timestamp: str | None = None, accion_id: str | None = None): None smtplib STARTTLS :587

«module»
iam_providers

+ IAM_MOCK_URL = env | "http://127.0.0.1:8002"
+ IAM_TIMEOUT_SEC = 1.0
+ KEYCLOAK_BASE_URL = env | "http://127.0.0.1:8080"
+ KEYCLOAK_REALM, KEYCLOAK_CLIENT_ID, KEYCLOAK_CLIENT_SECRET: env
+ KEYCLOAK_TIMEOUT_SEC = 1.0
+ KEYCLOAK_TOKEN_REFRESH_MARGIN_SEC = 30
+ ATRIBUTO_RIESGO = "ueba_risk_level"
+ GRAPH_BASE_URL = "https://graph.microsoft.com/v1.0"
+ ENTRA_LOGIN_URL = "https://login.microsoftonline.com"
+ ENTRA_TIMEOUT_SEC = 1.0
+ ENTRA_ATRIBUTO_RIESGO = "extensionAttribute1"
+ IAM_PROVIDER = env | "none"
+ PROVEEDORES: dict[str, Callable[[dict], IAMProvider]] {"mock", "keycloak", "entra"}
- _iam_provider: IAMProvider | None «singleton»
- _config_redis: list[dict] | None

- _cfg_default(): _KcCfg
- _keycloak_token_cache_key(cfg: _KcCfg | None): str
- _get_cached_token(cfg: _KcCfg | None): str | None
- _store_cached_token(access_token: str, expires_in: float, cfg: _KcCfg | None): None
- _fetch_service_token(cfg: _KcCfg | None): tuple[str, float] POST /realms/{realm}/protocol/openid-connect/token
- _get_service_token(cfg: _KcCfg | None): str
- _entra_token_cache_key(cfg: _EntraCfg): str
- _fetch_entra_token(cfg: _EntraCfg): tuple[str, float] POST /{tenant}/oauth2/v2.0/token
- _get_entra_token(cfg: _EntraCfg): str
- _build_provider(spec: str): IAMProvider mock | keycloak[:N] | entra:N
+ proveedores_entorno(): list[dict]
- _config_redis_items(): list[dict]
+ reset_provider(): None
+ iam_habilitado(): bool
- _from_item(item: dict): IAMProvider
+ get_iam_provider(): IAMProvider
+ probar_proveedores(): list[dict] {proveedor, ok, ms, error}

«module»
dedup_service

+ DEDUP_TTL_SECONDS = 21600 (6 h)
+ CAMPOS_HASH = ("uid", "time", "type", "params", "location", "role", "isLocalIP")

- _dedup_key(hash_hex: str): str
+ compute_event_hash(event: dict): str SHA-256
+ get_cached_response(hash_hex: str): dict | None
+ store_response(hash_hex: str, response: dict): None

Dependencias en otras láminas
iam_service, iam_providers, auth, dedup_service → redis_state (lámina A)
iam_providers → config_store.get_iam_config (lámina D)
notifications → artifacts (B), config_store (D), redis_state (A)
hybrid_service (B) invoca iam_service y notifications; routes (D) invoca auth y dedup_service